Titotto Paolo 895227

REPORT PROGETTO TAW

ARCHITETTURA DEL SISTEMA:

L'architettura del sistema prevede l'utilizzo di 3 principali componenti:

- Database → MongoDB
- Backend → Express - Node.js
- Frontend → Angular

Il database che è stato utilizzato per il progetto è MongoDB, si tratta di un database NoSQL, a differenza di PostrgeSQL, e anziché le tabelle utilizza i documenti per il salvataggio dei dati. Al suo interno sono presenti una serie di collezioni contenenti i dati principali per il funzionamento dell'applicazione.

Il backend utilizza Express, un modulo di Node.js che fornisce una serie di middleware per la costruzione di applicazioni web. Alcuni esempi possono essere:

- la gestione delle sessioni
- la compressione dei dati e gestione degli errori che possono accadere durante l'esecuzione.

Consente inoltre di risolvere alcuni problemi, ad esmpio la gestione delle rotte tra i vari endpoint dell'API e attraverso la libreria Mongoose è in grado di poter interagire con il database MongoDB.

Viene utilizzata anche la funzione cors() che consente di poter comunicare con diversi container connessi alla stessa rete in Docker. L'utilizzo di questo software è stato deciso per poter ottenere lo stesso risultato anche se il progetto venisse utilizzato in computer, sistemi operativi e architetture diverse rispetto a quello in cui è stato sviluppato, ovviando così qualsiasi problema di compatibilità.

Il frontend è stato strutturato tramite il framework Angular, molto utilizzato nello sviluppo di applicazioni SPA lato client. È basato sul linguaggio Typescript e utilizza il modello Model-View-Controller MVC che consente di separare la logica dalla presentazione delle informazioni.

Permette la visualizzazione delle informazione attraverso un browser in comunicazione con il backend. Il risultato sono dei file in formato JSON.

Esso è suddiviso in services, contenenti le richieste relative al backend, e components, attraverso i quali si può visualizzare a schemro i dati risultati dalle richieste effettuate dal service.

DATA MODEL:

Le collections create per lo sviluppo di questa web app sono le seguenti:

· **Users** Contengono tutti gli utenti, compresi compagnie aeree e amministratore.

I campi presenti in ogni suo documento sono:

1. _id : ObjectId (univoco)
2. Mail : string (univoca)
3. Username : string. Si riferisce al nome dell'utente
4. Password : string
5. passportID: string
6. roles : string. Indica il suolo di tale utente, può essere admin, passanger oppurre airline
7. tickets : Tickets [ ]

Titotto Paolo 895227

· **Airlines** Si riferiscono alle compagnie aeree.

All'interno di tale documento sono presenti i seguenti dati:

1. _signature : string (univoca) corrisponde alla sigla della compagnia aerea
2. Name : string. Nome assegnato a tale compagnia, scelto in fase di registrazione dall'admin ma sempre modificabile da parte dell'utente
3. Revenue : number. Indica il fatturato generato dalla vendita dei biglietti.
4. Mail : string usata come collegamento con gli utenti
5. mustChangePassword : boolean indica se è necessario cambiare la password
6. aircrafts : Aircrafts [ ]

· **Aircrafts** Contengono tutti gli aerei delle varie compagnie.

I suoi campi sono:

1. _numAircraft : string (univoco)
2. Seats: Seats[ ]
3. Modello : string

· **Seats** Contengono le caratteristiche dei posti per ciascun aereo

Il documento ha le seguenti caratteristiche:

1. _numSeat : string (univoco)
2. Class : string
3. Legroom : boolean. Indica la presenza dello spazio per le gambe maggiorato.
4. isFree : boolean. Indica se il posto è libero se assume valore True, mentre risulta occupato con il valore False.

· **Airports** Contengono tutti gli aeroporti.

I suoi campi sono:

1. _code : string (univoco) codice per identificare l'aeroporto
2. Name : string Sigla identificativa dell'aeroporto (esempio VCE, FCO, JFK)
3. City : string Città a cui appartiene l'aeroporto

Titotto Paolo 895227

· **Tickets** Contiene i biglietti che le compagnie aeree rendono disponibili alla vendita e acquistati dai passeggeri

Le informazioni contenute al suo interno sono:

1. _code : string (univoco)

2. User : ObjectId. Permette il collegamento con l'utente; si rivelerà utile al momento in cui il biglietto viene acquistato.

3. Seat : string. Indica il posto assegnato a quel biglietto

4. Flight : string. Si riferisce all codice del volo a cui è assegnato il biglietto

5. Price : number. Corrisponde all prezzo del biglietto

6. numBag : number. Indica il numero dei bagagli

· **Routes** Contengono tutte le rotte previste per i voli.

I suoi campi sono:

1. _id : ObjectId (univoco). Identifica la rotta specifica

2. Destination : string. Indica la destinazione

3. Departure : string. Indica la partenza

4. LayoverFuel : string [ ]. Si riferisce ai possibili aeroporti in cui viene eseguito uno scalo durante il tragitto selezionato

· **Flights** Al suo interno sono presenti tutti i voli decisi dalle varie compagnie aeree

Le informazioni sono:

1. _flightCode : string (univoco). Indica il codice del volo, solitamente identificati con la signature della compagnia aerea e una sequenza di uno o quattro numeri.

2. dataOra : Date. Si riferisce alla data e l'ora di partenza del volo secondo lo standard ...

3. route : ObjectId. Riferimento alla rotta seguita dal volo

4. aircraft : string. Riferimento all'identificativo dell'aereo che effettuerà il volo

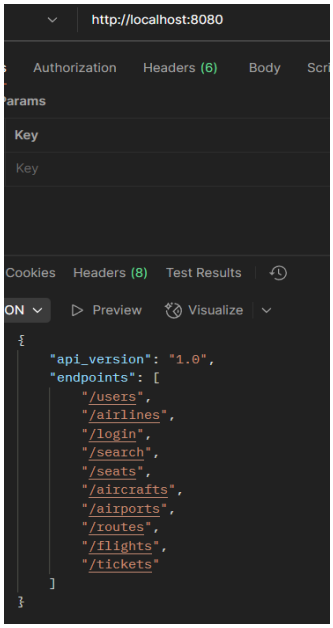
5. duration : number. Indica la durata del volo in minuti

Titotto Paolo 895227

REST API:

Per effettuare i testi è stato utilizzato un tool apposito denominato Postman.

L'API utilizza i seguenti endpoints preceduti da /api/v1 per indicare che si tratta di una API e la sua relativa versione:

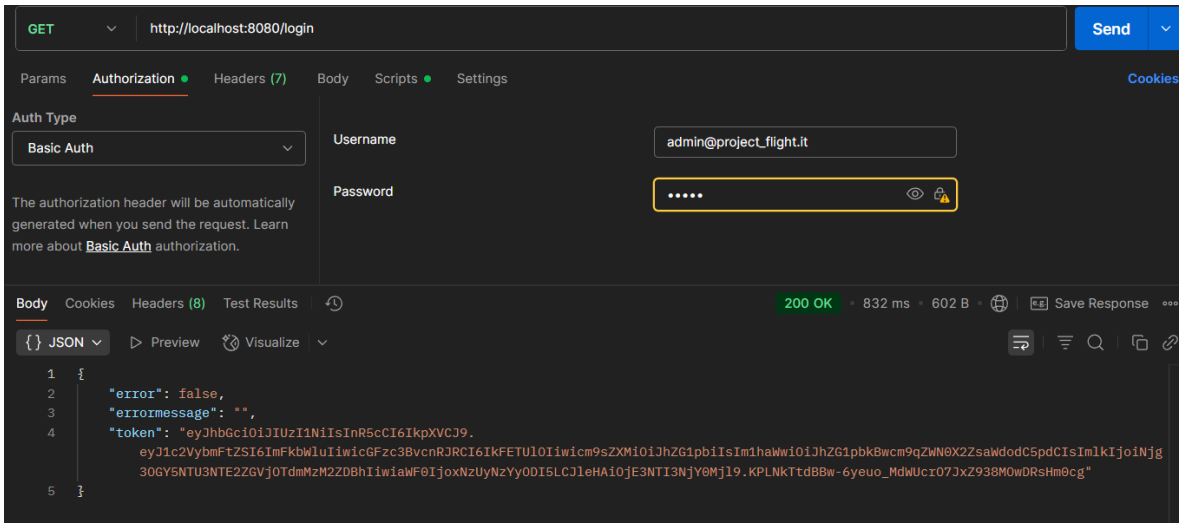


· /login

È una richiesta di tipo GET necessaria per l'autenticazione dei vari utenti. I parametri necessari che devono essere passati sono la mail e la password.

Esso utilizza un'autenticazione di tipo Basic, in cui vengono controllate le credenziali. In caso siano corrette vengono codificati dei dati all'interno di una stringa detto JSON Web Token.

Questo token verrà firmato e inviato all'utente per poter essere utilizzato nelle successive richieste in cui è prevista l'autenticazione, in questo caso prende il nome di Bearer Token.



· **/users**

- Se si tratta di una richiesta di tipo POST, viene utilizzata per la registrazione di un nuovo utente, viene impedita lato frontend se il nuovo utente ha come ruolo “airline”.

Se si tratta di un utente con ruolo passenger i parametri saranno quelli di users, di conseguenza la mail, lo username, la password, il passportID e il ruolo.

- Se è una richiesta di tipo GET non ha la necessità di alcun parametro, ritorna tutti gli utenti il cui ruolo è passenger e può essere eseguita solo se si è un admin. Per effettuare questo filtro viene utilizzata la mail che è univoca.

· **/users/:mail**

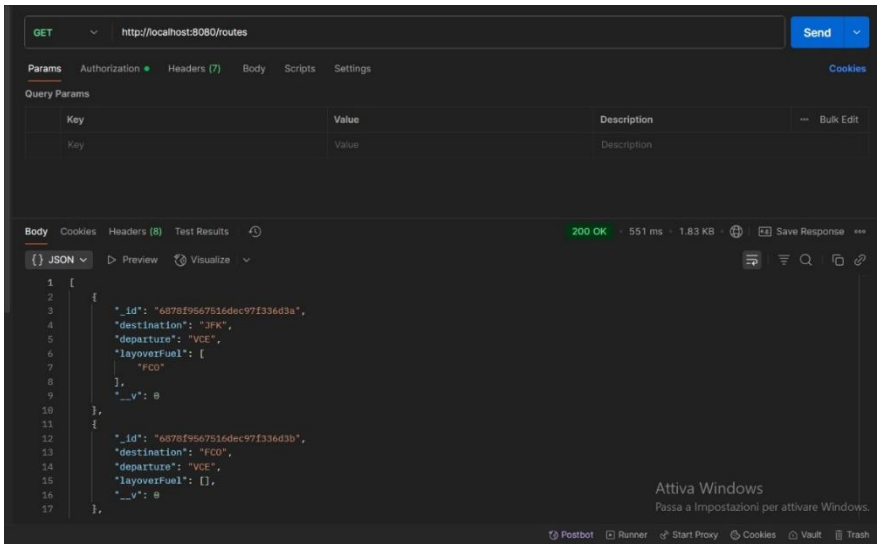
È una richiesta di tipo DELETE necessaria per l’eliminazione di un utente. Viene eseguita passando come parametro la mail dell’utente da eliminare. Il suo utilizzo è a disposizione esclusiva dell’admin.

· **/users/:id**

Richiesta di tipo PUT utilizzata per modificare i dati dell’utente. I parametri in input sono l’id dell’utente e i parametri che si intende modificare quali password, username, mail, passportID.

· **/routes**

- Se si tratta di una richiesta di tipo GET, questa ritorna tutte le rotte presenti nel database. Non prende nessun parametro.



- Se, invece, viene invocata la richiesta di tipo POST serve per aggiungere una nuova rotta. Eseguibile esclusivamente dopo aver effettuato l’accesso alla propria area riservata con un utente avente ruolo “airline”. I parametri richiesti sono: la partenza, la destinazione e l’eventuale scalo.

· **/routes/:id**

- Se una richiesta è di tipo PUT allora può essere utilizzata per apportare modifiche si dati delle rotte, in particolare è possibile modificare l’aeroporto partenza, quello di destinazione e l’eventuale scalo. Per indicare correttamente l’aeroporto vengono utilizzate le rispettive sigle.

- Se è una richiesta di tipo DELETE serve per la cancellazione delle rotte, può essere effettuata solo dalle compagnie aeree e prende come parametro l’id della rotta da cancellare.

· **/airlines:/mail**

È di tipo GET attraverso questa richiesta si ottengono tutte le informazioni sulla compagnia aerea relative alla mail passata come parametro. Eseguibile esclusivamente dalla compagnia aerea.

· **/airlines**

- Se è una richiesta di tipo GET, eseguibile solo dall'admin e viene utilizzata prt ritornare le informazioni di tutte le compagnie aeree presenti nel database. Non ha alcun parametro in input in quanto ritorna tutte le compagnie.

- Se è una richiesta di tipo POST è a disposizione solo dall'admin e inserisce una nuova compagnia aerea. I parametri necessari sono _signature, nome, mail e una password temporanea, che verrà necessariamente cambiata al primo login effettuato da tale utente.

· **/airlines/:mail/aircrafts**

È una richiesta di tipo GET, eseguibile solo dalla compagnia aerea e restituisce tutti i velivoli appartenenti alla compagnia che ha appena effettuato il login, il parametro è per l'appunto l'indirizzo mail.

· **/airlines/:id**

- Se è una richiesta di tipo PUT serve per modificare determinate informazioni della compagnia aerea, quali nome, revenue e gli aerei. Può essere eseguita da un utente il cui ruolo è airline.

- la richiesta ha tipo DELETE. Viene utilizzata per rimuovere una compagnia aerea. Essa può essere eseguita solo dall'amministratore e il parametro necessario è la _signature della compagnia aerea.

· **/airlines/:id/setup**

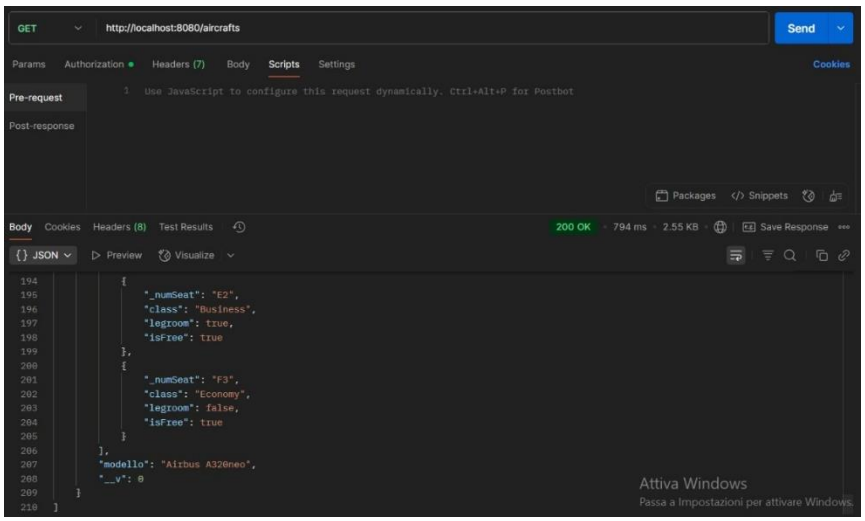
Si tratta di una richiesta PUT e serve per il cambio password al momento del primo login da parte delle compagnie aeree.

L'input necessario è la nuova passoword che deve essere inserita per poter confermare la modifica.

· **/aircrafts**

- Se è una richiesta di tipo POST serve per aggiungere nuovi aerei, eseguibile esclusivamente dalla compagnia. I parametri in input sono il numero dell'aereo, un array di posti e il modello.

- Se la richiesta ha tipo GET ritorna tutti gli aerei presenti nel database. Non necessita di nessun parametro in input.



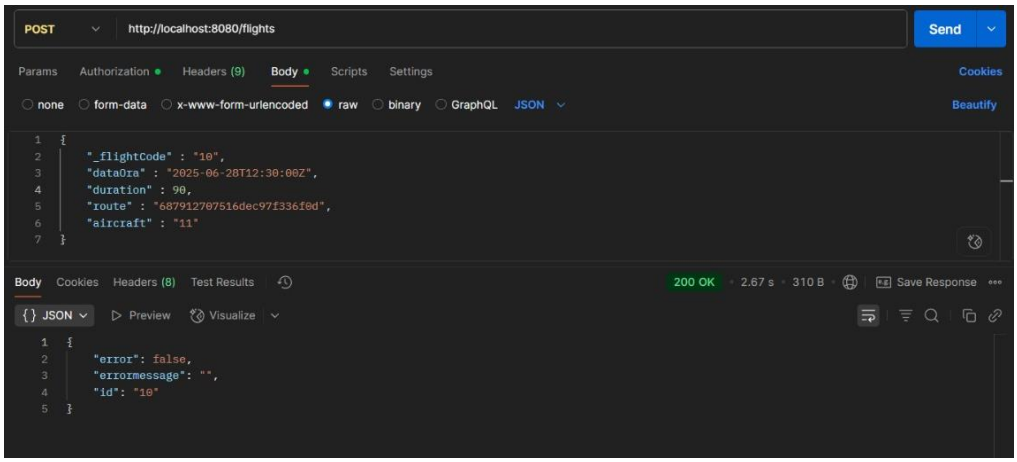
· **/aircrafts/:id**

- Se si tratta di una PUT, questa serve per modificare alcuni dati riguardanti l'aereo. I dati modificabili sono: i posti presenti, il modello e il numero dell'aereo da modificare. Può essere eseguita solo dalla compagnia aerea.

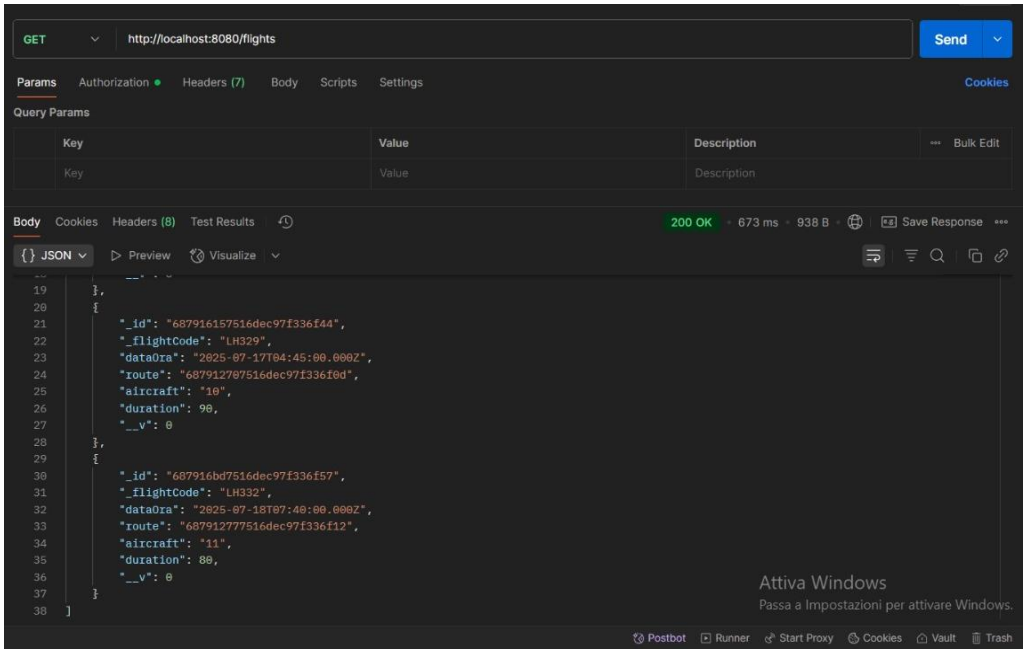
- Se è una richiesta utilizzata per l'eliminazione ha come tipo DELETE. Necessita esclusivamente del numero dell'aereo da cancellare. Per poterla eseguire bisogna essere necessariamente una compagnia.

· **/flights**

- Se si tratta di una richiesta POST il risultato è lsa possibilità di aggiungere un nuovo volo per una determinata rotta già esistente, può essere eseguito solo da un utente con ruolo airline. I parametri necessari per il suo corretto funzionamento sono: _flightCode è un codice che identifica il singolo volo, dataOra indica la data e l'ora di partenza del volo, aircraft indica il numero dell'aereo assegnato a quel volo e la durata in minuti.



- Se è una richiesta di tipo GET si ottiene come risultato tutti i voli che tale compagnia compie con gli aerei a sua disposizione. Non necessita di parametri in input.



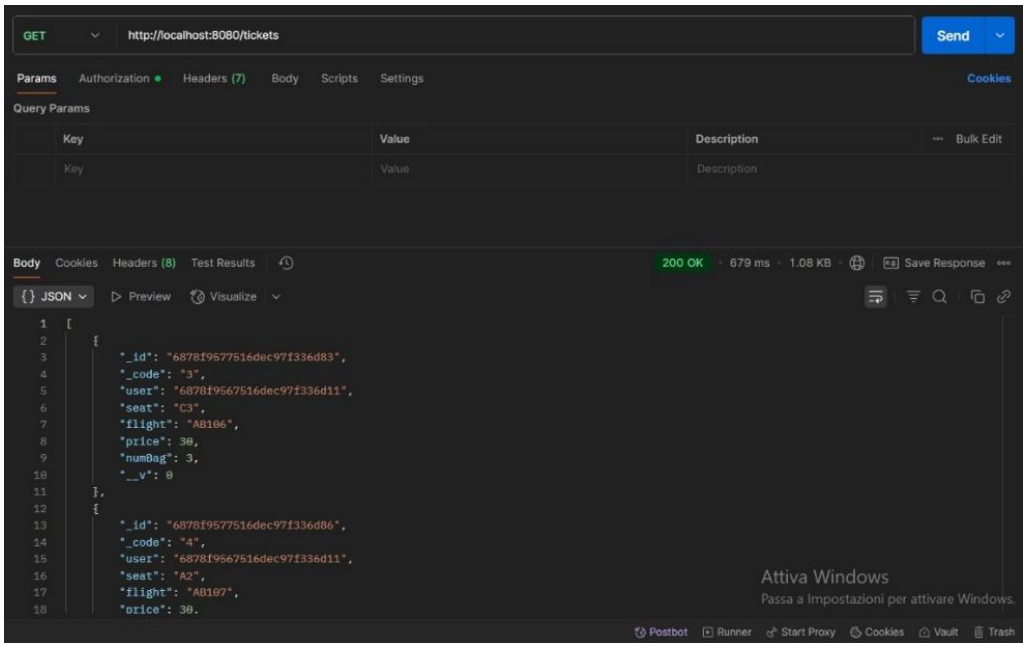
· **/flights/:id**

- Se è una richiesta di tipo PUT serve per modificare determinate informazioni riguardanti i voli già presenti, quali dataOra, gli aircrafts, la rotta e la durata del volo. L'unico utente a poterla eseguire è quello che ha ruolo "airline".

- Se è una richiesta di tipo DELETE serve per cancellare un volo già inserito. Può essere eseguibile solo dalla compagnia aerea. Necessita esclusivamente del codice del volo da dover eliminare per procedere in modo corretto con l'eliminazione,

· **/tickets**

- Se è una richiesta di tipo GET ritorna tutti i biglietti associati all'utente che ha fatto il login. Può essere eseguita dai passeggeri e dalle compagnie aeree.



- Se è una richiesta di tipo POST può essere eseguita da qualsiasi utente con ruolo passeggero o compagnia aerea.

Nel primo caso si procede con la rimozione del biglietto dall'array dei tickets della compagnia aerea e viene aggiunto a quello dell'utente che lo ha acquistato. Inoltre il posto assegnato al biglietto viene marcato come occupato tramite la variabile booleana isFree.

Viene utilizzata solo dopo aver effettuato la ricerca e il login.

Se si tratta, invece, del secondo scenario, il biglietto viene aggiunto alla collection dei tickets e nell'array della compagnia rimuovendolo da quello del passeggero.

I parametri di input necessari sono: _code che indica il codice del biglietto, user indica l'utente a cui è assegnato, seat indica il posto, flight indica il volo, price indica il prezzo del biglietto e numBag indica il numero dei bagagli ammessi.

· **/tickets/:id**

- Se è una richiesta di tipo PUT viene utilizzata per modificare alcuni dati del biglietto. Può essere eseguita solo dalla compagnia aerea. I parametri di input sono il codice del biglietto da cui si possono ricavare le informazioni di seat, numBag e price che si possono modificare in caso di necessità.

- Se si tratta di una DELETE può essere eseguita dalla compagnia aerea e dal passeggero.

Nel primo caso si tratta della cancellazione assoluta del biglietto passando come input il codice del biglietto da rimuovere.

Nel secondo caso, invece, il biglietto non sarà più associato al passeggero ma ritorna nell'array della compagnia. Il posto assegnato torna ad essere libero.

· **/airports**

- Se è una richiesta GET vengono ritornati tutti gli aeroporti presenti nel database. Non necessita di alcun parametro in input. Viene eseguita solo dall'admin.

- Se è una richiesta POST può essere eseguita solo dall'admin e inserisce un nuovo aeroporto nel database. I parametri di input sono: _code è un codice che identifica l'aeroporto, name che corrisponde alla sigla dell'aeroporto e la città.

· **/airports/:id**

- Se è una richiesta di tipo PUT è a esclusiva disposizione dell'admin. Serve per modificare la città e il nome dell'aeroporto. I parametri necessari sono: il codice dell'aeroporto.

- Se è una richiesta di tipo DELETE è eseguibile solo dall'admin e serve per cancellare un aeroporto. Il parametro necessario è il codice dell'aeroporto da eliminare.

· **/seats**

- Se è una richiesta di tipo GET restituisce tutti i posti assegnati ai vari aerei. Non necessita di nessun parametro in input.

- Se è una richiesta di tipo POST serve per creare nuovi posti, può essere eseguita solo dalla compagnia aerea. I parametri necessari sono: _numSeat indica il numero del posto, class indica la classe, legroom indica se è previsto spazio per le gambe e isFree che indica se il posto è libero.

· **/seats/:id**

- Se una richiesta di tipo PUT è eseguibile esclusivamente dalla compagnia aerea, vierne utilizzata per modificare alcune informazioni sul posto passato come input. Altri parametri di input sono la classe, legroom e isFree.

· **/search**

È una richiesta di tipo GET, può essere eseguita da chiunque, infatti non prevede l'autenticazione.

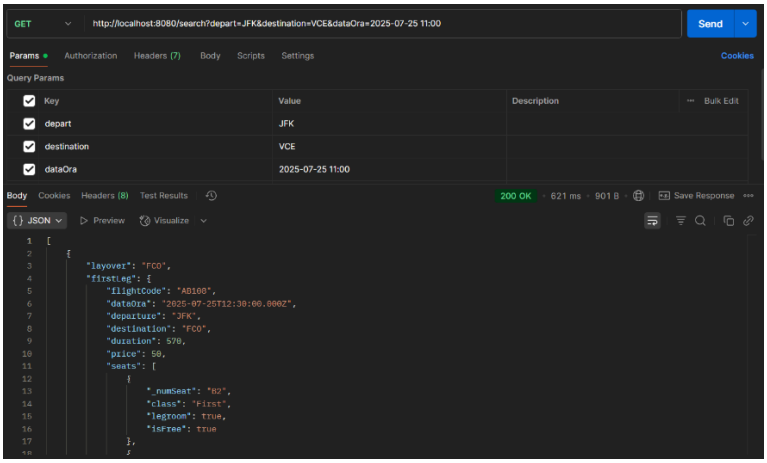
I parametri di input sono: la partenza, la destinazione, data e ora di partenza.

In caso di voli diretti restituisce: il codice del biglietto, la data e ora di partenza, partenza, destinazione, durata, il prezzo in base alla classe di ciascun posto libero e i posti liberi.

In caso di voli con scalo restituisce il biglietto per il volo dalla partenza fino allo scalo e quello per il volo dallo scalo fino alla destinazione.

I biglietti restituiti in output sono in ordine crescente rispetto al numero di posto assegnato al singolo biglietto.

Titotto Paolo 895227




```
[
    {
        "layover": "FCO",
        "firstLeg": {
            "flightCode": "AB168",
            "dataOra": "2025-07-25T12:30:00.000Z",
            "departure": "JFK",
            "destination": "FCO",
            "duration": 570,
            "price": 50,
            "seats": [
                {
                    "_numSeat": "82",
                    "class": "First",
                    "legroom": true,
                    "isFree": true
                },
                {
                    "_numSeat": "B3",
                    "class": "Economy",
                    "legroom": false,
                    "isFree": true
                }
            ]
        },
        "secondLeg": {
            "flightCode": "AB109",
            "dataOra": "2025-07-26T00:00:00.000Z",
            "departure": "FCO",
            "destination": "VCE",
            "duration": 70,
            "price": 50,
            "seats": [
                {
                    "_numSeat": "C1",
                    "class": "First",
                    "legroom": true,
                    "isFree": true
                },
                {
                    "_numSeat": "C2",
                    "class": "Economy",
                    "legroom": true,
                    "isFree": true
                },
                {
                    "_numSeat": "C3",
                    "class": "Business",
                    "legroom": true,
                    "isFree": true
                }
            ]
        }
    }
]
```

Titotto Paolo 895227

DESCRIZIONE FRONTEND:

Il frontend è stato sviluppato utilizzando il framework Angular ed è stato suddiviso tra components e services.

Per ogni collection presente in MongoDB è stato creato un service che si occupa delle varie chiamate all'API e un component per poter visualizzare le varie informazioni.

· Change-password: component che si si tratta della pagine relativa al cambio password per il primo login delle compagnie aree

· Admin: component e service per interagire con le operazioni eseguibili dall'admin

· User-login: component viene utilizzato per la gestione del login

· User-signup: component per la gestione della registrazione di un nuovo utente avente tipo "passenger".

· Passengers: component e service che permette di interagire con tutte le funzionalità attribuite ai passeggeri presenti nella dashboard.

· Dashboard: component che in base al ruolo dell'utente che ha eseguito il login mostra una pagina differente contenente tutte le informazioni ad esso relative, con tutte le chiamate al backend associate.

· Aircrafts: service per gestire le richieste con il backend, mentre il componente è integrato all'interno di airlines.

· Airlines: component e service che permettono di interagire con le funzionalità previste per le compagnie aeree.

· Airports: service che permette di interagire con le richieste degli aeroporti, il component è integrato all'interno di admin

· Flights: component e service per la gestione dei voli. A disposizione esclusiva delle compagnie.

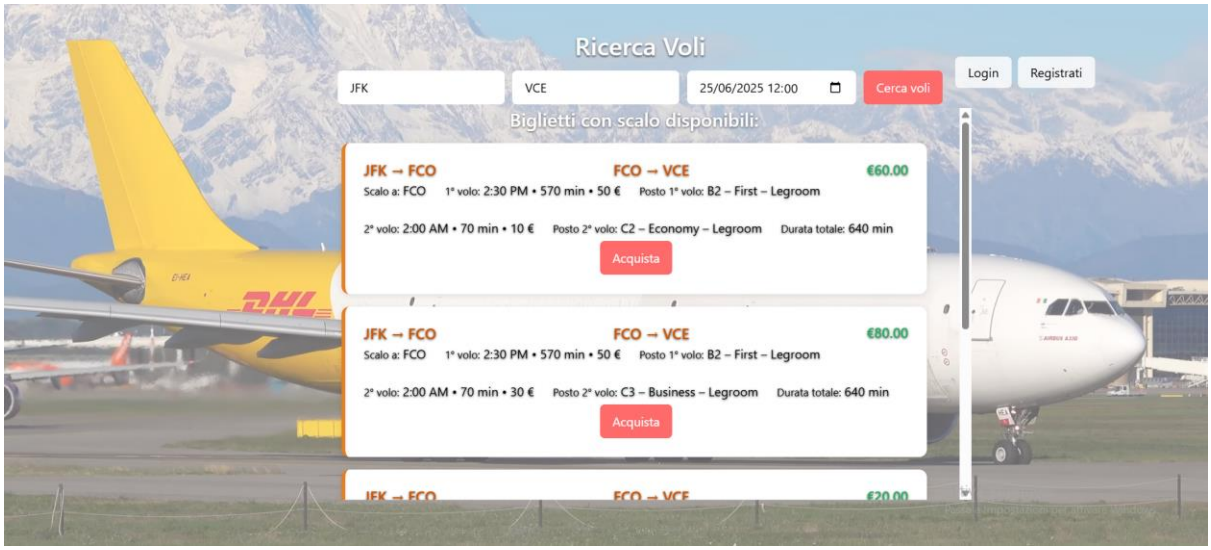
· Routes: component e service per la gestione delle rotte per i voli presenti o per aggiungerne di nuovi.

· Search: component e service per la ricerca dei biglietti e per procedere all'acquisto

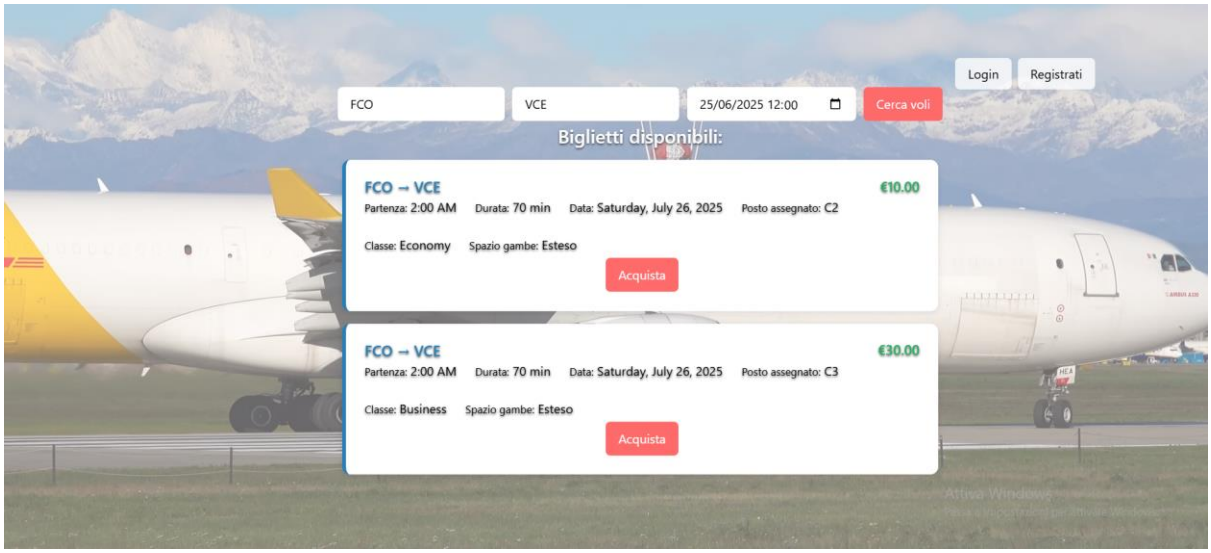
· Tickets: service per l'invio delle richeste al backend, ma il component è integrato in airlines e passengers

Le routes previste in Angular sono:

· **/search**: Sono presentii 2 pulsanti uno relativo al login e uno per la registrazioni che portano alle relative pagine e il pulsante acquista che riconduce al login. Per effettuare la ricerca si devono inserire le sigle degli aereoporti di partenza e arrivo con la data rispettiva.



Titotto Paolo 895227



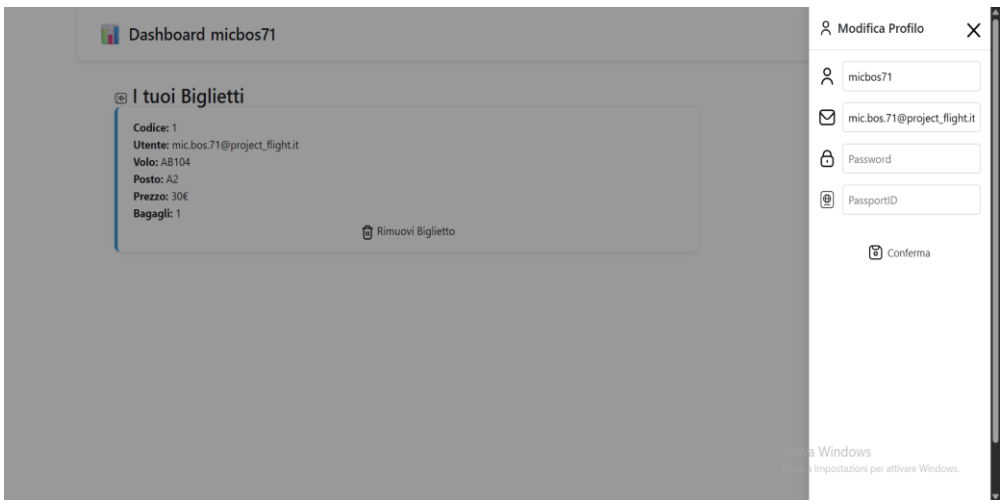
· **/login**: Dopo il login si viene indirizzati alla pagina della dashboard da cui è possibile gestire tutte le funzionalità a seconda del proprio ruolo. Per una compagnia aerea che esegue per la prima volta il login viene inizialmente ricondotta alla pagina per il cambio password.

· **/signup**: Pagina utilizzata per effettuare nuove registrazioni, esclusivamente per utenti con ruolo di passeggeri. Una volta registrati si viene indirizzati alla pagina di login

· **/change-password**: Una volta cliccato il pulsante cambia password si viene reindirizzati alla propria dashboard.

· **/dashboard**: A login avvenuto si effettua l'accesso alla dashboard che contiene le funzioni principali per ogni utente:

o Se l'utente è passenger (passeggero) allora può solo visualizzare ed eliminare i biglietti acquistati e modificare i dati inseriti al momento della registrazione.



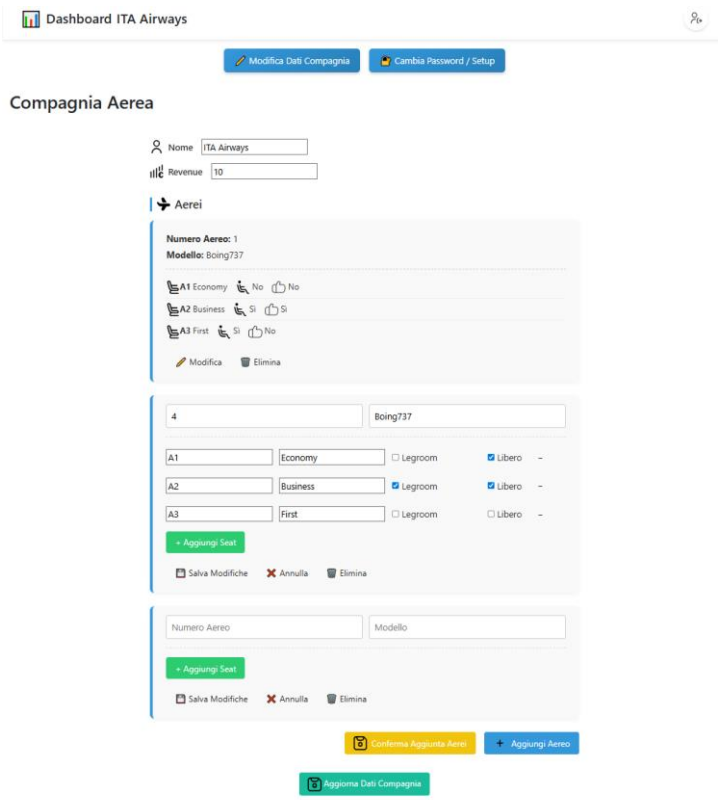
- Se l’utente è airline (compagnia aerea) allora può:

§ Aggiungere, modificare ed eliminare velivoli;

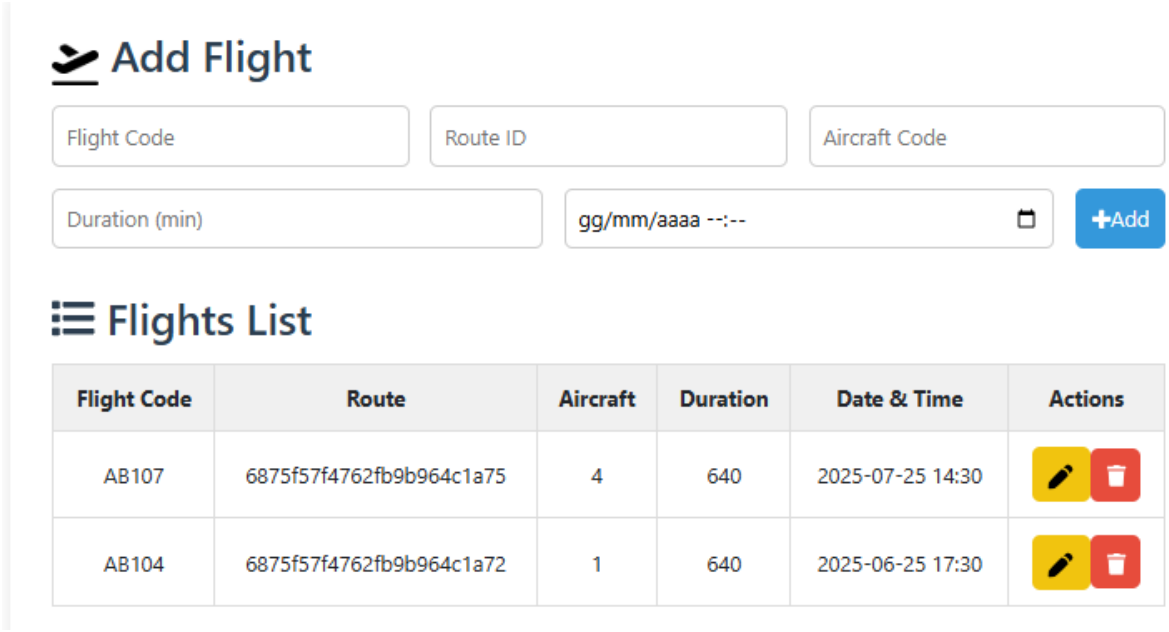
§ Aggiungere, modificare ed eliminare voli;

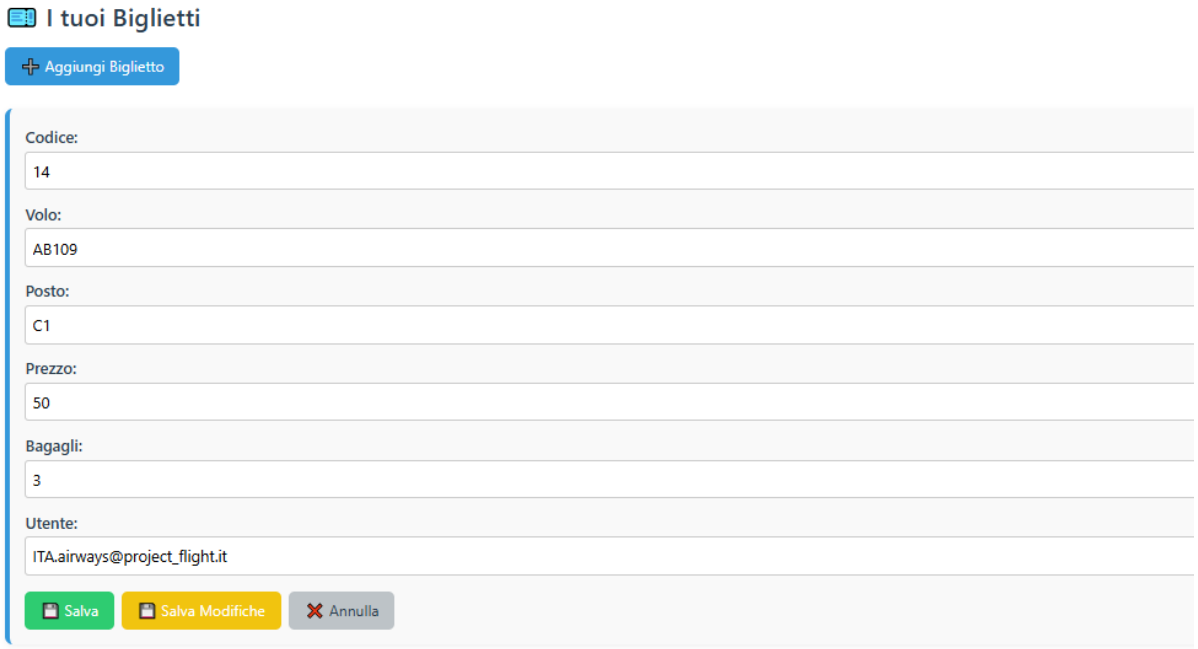
§ Aggiungere, modificare ed eliminare biglietti acquistabili, necesario creare uno per ogni posto dell’aeromobile;

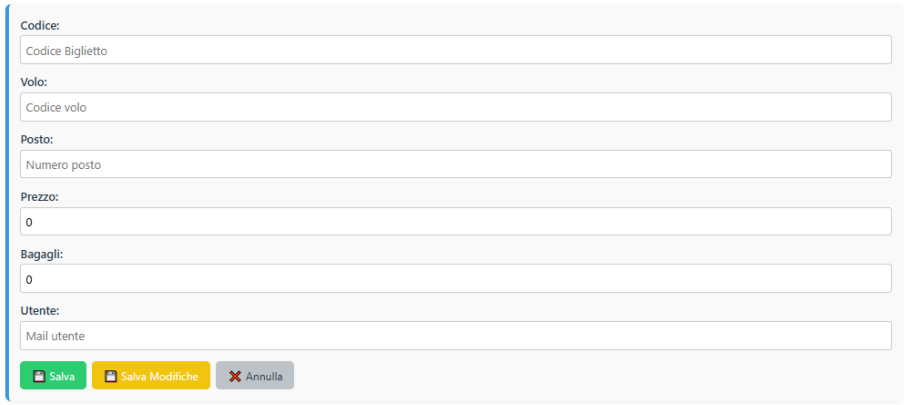
§ Aggiungere, modificare ed eliminare rotte.

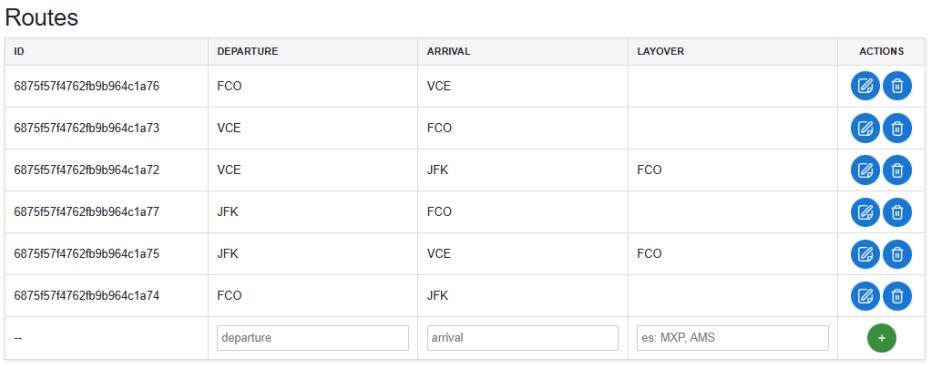


Titotto Paolo 895227



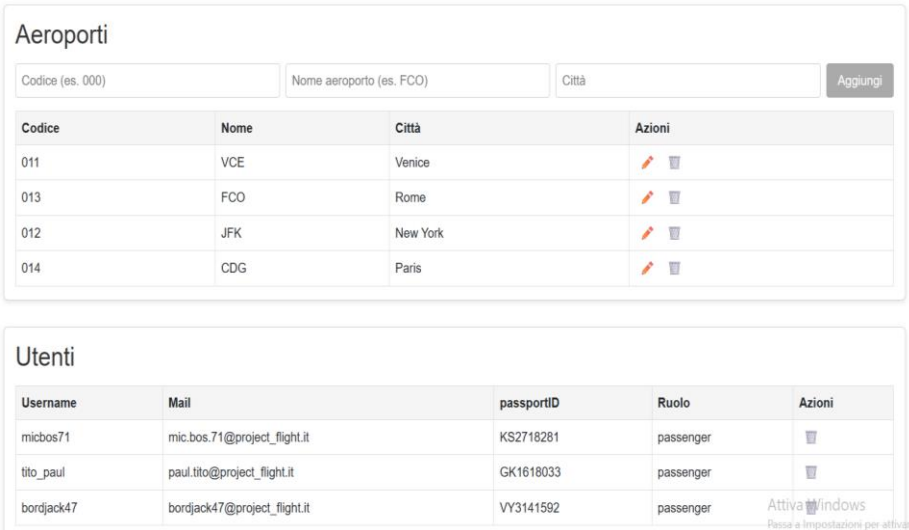






- Se l'utente è admin allora può:

§ Aggiungere, modificare ed eliminare aeroporti;



Titotto Paolo 895227

§ Invitare le compagnie aeree tramite un form tramite il quale fornisce a quest'ultime una password temporanea.

